

WEEK 8 Practical OSSP

PART 1 — Dynamic Memory Allocation + Valgrind

1. Create the C program

```
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ nano dynamic_memory.c
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ gcc -Wall -g dynamic_memory.c -o dynamic_memory
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ ./dynamic_memory
malloc(): 10 20 30
calloc(): 0 0 0
realloc(): 10 20 30 40 50
Memory successfully freed.
```

2. Check for memory leaks

```
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ valgrind --version
valgrind-3.22.0
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ valgrind --leak-check=full ./dynamic_memory
==112== Memcheck, a memory error detector
==112== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==112== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==112== Command: ./dynamic_memory

==112== HEAP SUMMARY:
==112==      in use at exit: 0 bytes in 0 blocks
==112==    total heap usage: 4 allocs, 4 frees, 556 bytes allocated
==112==
==112== All heap blocks were freed -- no leaks are possible
==112==
==112== For lists of detected and suppressed errors, rerun with: -s
==112== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Summary:

A C program was developed using malloc(), calloc(), realloc() and free() for dynamic memory management. The program successfully allocated, initialized, resized and released memory. Valgrind was used to check the program for memory leaks, and the analysis showed that all allocated memory was properly freed with no errors.

PART 2 — Detecting a Memory Leak using Valgrind

1. Create a program with a deliberate leak, Compile and Run

```
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ nano memory_leak.c
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ gcc -Wall -g memory_leak.c -o memory_leak
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ ./memory_leak
Allocated values: 10 20 30 40 50
```

3. Detect the leak

```
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ valgrind --leak-check=full ./memory_leak
==96== Memcheck, a memory error detector
==96== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==96== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==96== Command: ./memory_leak
==96==
==96== error calling PR_SET_PTRACER, vgdb might block
Allocated values: 10 20 30 40 50
==96==
==96== LEAK SUMMARY:
==96==    definitely lost: 20 bytes in 1 blocks
==96==    indirectly lost: 0 bytes in 0 blocks
==96==    possibly lost: 0 bytes in 0 blocks
==96==    still reachable: 0 bytes in 0 blocks
==96==    suppressed: 0 bytes in 0 blocks
==96==
==96== For lists of detected and suppressed errors, rerun with: -s
==96== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

4. Fix the leak

```
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ nano memory_leak.c
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ gcc -Wall -g memory_leak.c -o memory_leak
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ valgrind --leak-check=full ./memory_leak
==103== Memcheck, a memory error detector
==103== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==103== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==103== Command: ./memory_leak
==103==
==103== error calling PR_SET_PTRACER, vgdb might block
==103==
==103== HEAP SUMMARY:
==103==    in use at exit: 0 bytes in 0 blocks
==103==    total heap usage: 2 allocs, 2 frees, 532 bytes allocated
==103==
==103== All heap blocks were freed -- no leaks are possible
==103==
==103== For lists of detected and suppressed errors, rerun with: -s
==103== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Summary:

In this part, a C program was developed to demonstrate dynamic memory allocation and memory leaks. Valgrind was used to detect an intentionally unfreed memory block, which was reported as a memory leak. The leak was then fixed using `free()`, and Valgrind confirmed that all allocated memory was properly released.

PART 3 — Copy-on-Write using `fork()`

1. Create the C program, Compile and Run the program

```
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ nano cow.c
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ gcc -Wall -g cow.c -o cow
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ ./cow
Before fork: data = 100
Child before modification: data = 100
Child after modification: data = 200
Parent after child modification: data = 100
```

2. Demonstrate the memory addresses

```
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ gcc -Wall -g cow.c -o cow
ashborn@DESKTOP-I4G5Q4N:~/week8_prac$ ./cow
Before fork: data = 100, address = 0x7ffffef1c02a0
Child before modification: data = 100, address = 0x7ffffef1c02a0
Child after modification: data = 200, address = 0x7ffffef1c02a0
Parent after child modification: data = 100, address = 0x7ffffef1c02a0
```

Observation:

Before `fork()`, the dynamically allocated variable contained the value 100. After `fork()`, both the parent and child initially accessed the same value. When the child modified its value to 200, the parent's value remained 100, demonstrating Copy-on-Write behavior.